BSDA2001P: Introduction to DL and GenAI Project

Project Report Guidelines

This document outlines the structure and content required for your final project report. The report should be a comprehensive document detailing your work on the Kaggle competition, from data exploration to model evaluation and conclusion.

1. Abstract / Executive Summary

This report details the end-to-end development of a deep learning pipeline for the "Jan 2026 DLGenAI Project - Messy Mashup" Kaggle competition. The core challenge was classifying 30-second audio mashups into 10 musical genres, complicated by the injection of ESC-50 environmental noise and stem-mixing artifacts. To bridge the gap between clean training data and noisy test data, I employed a multi-stage approach. I synthesized a robust training set by mixing stems and injecting dynamic noise, utilized an Audio Spectrogram Transformer (AST) as the primary backbone, and leveraged advanced techniques like Test-Time Augmentation (TTA) and high-confidence Pseudo-Labeling.

To ensure architectural diversity, I experimented with three distinct models: a fine-tuned AST, a custom Deep CNN built entirely from scratch, and a pre-trained EfficientNet-B0 adapted for audio. Ultimately, overcoming severe overfitting challenges via a combined "Megaset" of 14,000+ augmented audio chunks allowed the AST pipeline to breach the 0.973+ Macro F1 threshold. Initial runs of the CNN and EfficientNet yielded scores of 0.90574 and 0.94974 respectively, demonstrating immense potential for a powerful future ensemble.

2. Introduction

Problem Statement: The competition tasked us with classifying highly distorted audio "mashups." Unlike standard audio classification, the test set did not consist of clean songs. Instead, it contained tracks built by summing isolated stems (drums, bass, vocals, other) and artificially overlaying environmental noise from the ESC-50 dataset. This created a massive "domain gap" between any standard musical dataset and the hidden test set.

Project Objective: My primary goal was to achieve a top-tier leaderboard score (>0.97 Macro F1) by mastering the intricacies of PyTorch Lightning, Hugging Face audio transformers, and advanced data augmentation. A secondary objective was to deeply understand the mechanics of overfitting and regularization, specifically learning how to balance heavy augmentations against dataset purity across drastically different neural architectures.

Report Structure: Section 3 details my data generation and handling. Section 4 adapts tokenization theory to audio spectrograms. Section 5 breaks down the three distinct

architectures modeled. Section 6 analyzes the comparative performance of these models, and Section 7 reflects on the challenges and future ensembling directions.

3. Dataset & Preprocessing

Dataset Description: The raw data consisted of isolated audio stems categorized into 10 genres (blues, classical, country, disco, hiphop, jazz, metal, pop, reggae, rock), alongside a repository of ESC-50 noise files.

Exploratory Data Analysis (EDA): Initial audio analysis revealed that the test tracks were exactly 30 seconds long, whereas model architectures typically digest ~10-second windows. Furthermore, calculating the Root Mean Square (RMS) energy of the tracks revealed highly variable Signal-to-Noise Ratios (SNR). Some tracks had noise floors that almost entirely masked the harmonic content of the music.

Data Preprocessing & "The Megaset": Instead of training on clean stems, I built a custom generation script to reconstruct "true mashups." The script summed the 4 stems of a song and injected ESC-50 noise at random SNRs (5 to 15).

To prevent the model from overfitting to a small dataset, I eventually constructed the "Megaset," which concatenated three data sources:

1. **Synthetic Baseline Data:** ~10,000 randomly mixed and augmented stem mashups.
2. **Pseudo-Labels:** 449 test-set audio files where an earlier version of my model predicted the genre with >95% confidence, added back into the training loop to teach the model the exact test distribution.
3. **Pure Mashups:** 3,000 high-fidelity chunks crafted directly from the 1,000 master songs.

Data Augmentation: I utilized torchaudio to apply **SpecAugment** (Time Masking and Frequency Masking), which randomly drops out blocks of the spectrogram to force the model to rely on multiple distinct frequency bands. I also utilized **Mixup** (alpha=0.2) to blend spectrograms and labels, acting as a powerful regularizer against overconfidence.

4. Feature Extraction & "Tokenization" Strategy

Note: While the rubric specifies text tokenizers (e.g., WordPiece/BERT), this project tackles an audio modality. In Vision/Audio Transformers, the equivalent to "Tokenization" is Spectrogram Patchification.

Primary "Tokenizer" Selection: I utilized the Hugging Face ASTFeatureExtractor associated with the MIT/ast-finetuned-audioset-10-10-0.4593 model.

- **The Process:** Raw 16kHz audio waveforms were converted using librosa into 128-bin Mel-spectrograms. The AST Feature Extractor acts as the "tokenizer" by taking this 2D

continuous spectrogram image and splitting it into a sequence of 16x16 overlapping patches (tokens).

- **Justification:** Patchifying a Mel-spectrogram allows the Transformer's self-attention mechanism to look at relationships between specific localized acoustic events regardless of where they occur in time.
- **Implementation:** The extraction pipeline dynamically handled varying audio lengths using padding and truncation to ensure a fixed sequence length of 1024 frames, perfectly matching the expected token sequence length of the pre-trained AST.

5. Modeling & Experimentation

5.1. Model 1: Fine-Tuned Audio Spectrogram Transformer (AST)

- **Architecture:** I utilized the Audio Spectrogram Transformer (Gong et al., 2021), an attention-based model without convolutions. I loaded the MIT/ast-finetuned-audioset weights from Hugging Face, stripped the original 527-class head, and attached a custom Feed-Forward Network (FFN) with LayerNorm, Dropout, and a 10-class output.
- **Salient Points:** The AST's global self-attention mechanism is unparalleled at capturing long-range temporal dependencies in audio.
- **Fine-Tuning Strategy:** I used a "Soft Freeze-Thaw" strategy via differential learning rates. The delicate pre-trained backbone was assigned a micro-learning rate (2e-5), while the newly initialized classification head was trained at 1e-4. I optimized using AdamW and a Cosine Annealing schedule.

5.2. Model 2: Custom Deep Learning Model (2D CNN from Scratch)

- **Architecture:** To satisfy the requirement for a custom-built model, I designed a 5-block Deep Convolutional Neural Network from scratch. Each block consisted of a 3x3 Conv2d layer, BatchNorm2d, a ReLU activation, and a 2x2 MaxPool2d. The network culminated in an AdaptiveAvgPool2d layer, flowing into a dense classifier with 50% dropout.
- **Salient Points:** While Transformers act as "global listeners," CNNs act as "local listeners." The 3x3 convolutional kernels act as surgical texture-finders, sliding across the spectrogram to identify highly localized frequency patterns (like the specific crunch of an electric guitar) while ignoring global background noise.

5.3. Model 3: EfficientNet-B0 (Adapted for Audio)

- **Architecture:** I implemented a pre-trained EfficientNet-B0 model using torchvision.models. Because EfficientNet expects 3-channel RGB imagery, I adapted the forward pass to dynamically duplicate the 1-channel grayscale Mel-spectrogram across three channels (x.repeat(1, 3, 1, 1)). The standard ImageNet classification head was replaced with a robust dropout layer (p=0.4) and a 10-class linear output.
- **Salient Points:** EfficientNet is historically dominant in audio Kaggle competitions due to its use of Depthwise Separable Convolutions. These layers are incredibly efficient at isolating horizontal and vertical frequency textures—perfect for separating harmonic

instruments from broadband ESC-50 environmental noise without exploding the parameter count.

6. Performance & Comparative Analysis

Evaluation Metrics: Models were evaluated primarily on **Macro F1-Score**, which is unforgiving toward class imbalances and requires the model to perform equally well across all 10 genres.

Comparative Report & Kaggle Performance:

Model Architecture	Training Strategy	Kaggle Public Leaderboard (Macro F1)
AST (Fine-Tuned)	Differential LRs, Megaset, Full Epochs	**0.97327**
EfficientNet-B0	ImageNet Weights, 3-Channel Adapted	**0.94974** *(Partial Training)*
Custom CNN	Built from Scratch	**0.90574** *(Partial Training)*

Analysis of Results: The **AST** remains the dominant architecture for this task. Its global attention mechanism excels at synthesizing the chaotic, 30-second mashups into a cohesive genre prediction, easily clearing the 0.97 hurdle.

The **EfficientNet-B0** and **Custom CNN** were deployed as secondary exploratory architectures and intentionally trained for fewer epochs to establish a baseline. Despite the shortened training cycle, the EfficientNet achieved a highly impressive 0.94974, proving that its Depthwise Separable Convolutions adapt beautifully to audio textures. The Custom CNN achieved a 0.90574; considering it had to learn the fundamental concepts of audio processing entirely from random initialization (without the benefit of pre-trained ImageNet or AudioSet weights), this is a massive success and validates the custom 5-block architecture.

BSDA2001P: Introduction to DL and GenAI Project

7. Conclusion & Future Work

Key Learnings: This project was a masterclass in the realities of applied Deep Learning. I learned that data volume and augmentation strategy are often more important than model architecture. My attempt to feed "pure" but limited data to a massive Transformer proved that without the regularization of Mixup and large datasets, large models will simply memorize the training set.

Challenges Faced: I encountered significant pipeline engineering challenges, including Kaggle DataLoader multiprocessing deadlocks (solved by explicitly setting num_workers=0 on CPU bottlenecks) and PyTorch 2.6 weights_only=True unpickling security errors when loading custom checkpoints.

Future Work (The Ensembling Strategy): The immediate next step is to resume training the Custom CNN and EfficientNet models for a full epoch cycle to maximize their individual F1 scores. Once fine-tuned, these models will serve as the perfect "local-listening" counterparts to the AST's global attention. Because CNNs and Transformers make fundamentally different mathematical errors, ensembling them by averaging their raw Test-Time Augmentation (TTA) prediction probabilities will drastically reduce variance. I am confident that a hard-voting ensemble of these three distinct architectures will allow the pipeline to shatter the 0.99 F1 ceiling.

8. References

1. Gong, Y., Chung, Y. A., & Glass, J. (2021). AST: Audio Spectrogram Transformer. *Interspeech 2021*.
2. Tan, M., & Le, Q. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. *ICML*.
3. McFee, B., et al. (2015). librosa: Audio and Music Signal Analysis in Python.
4. Hugging Face Model Hub. *MIT/ast-finetuned-audioset-10-10-0.4593*.
5. Falcon, W., & The PyTorch Lightning team. (2019). *PyTorch Lightning*.
6. Piczak, K. J. (2015). ESC: Dataset for Environmental Sound Classification.